HARNESS LAB · MÓDULO 2 · AULA 2.2

O hook que roda lint e teste *a cada edição*

Você pede, o Claude Code escreve o hook. Daí em diante, cada edição passa pelo lint, cada resposta termina com os testes dos arquivos que mudaram, e o erro volta sozinho para o modelo.

`PostToolUse` `Stop` `Edit|Write` `exit 2` `additionalContext` `stop_hook_active` `quality-gate.sh`

VOCÊ SAI COM

O `quality-gate.sh` ligado no seu projeto: lint a cada edição, testes no fim da resposta, e ambiente fora do ar avisado sem prender o modelo num loop.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

A aula abre com dois avisos. Primeiro: cada hook é da sua stack. Na gravação, Laravel, com Pest nos testes e Pint no lint. Segundo: ninguém escreve o `settings.json` nem o script à mão. O mais importante hoje é saber pedir. O pedido foi este:

> **O PEDIDO DA AULA**
>
> "Vamos construir um hook ou dois que, ao editar ou escrever arquivos no projeto, execute testes, bem como execute lint no projeto, para manter a qualidade das entregas e o padrão de código. Se algum deles falhar, o modelo deve ser avisado."

O Claude Code devolveu dois arquivos. No `.claude/settings.json`: a chave `hooks`, o evento `PostToolUse` (depois que uma ferramenta roda: o Claude Code tem as ferramentas, o modelo decide quando usar), o matcher `Write|Edit|MultiEdit` e o comando para `.claude/hooks/quality-gate.sh`. Um segundo evento, `Stop`, roda os testes no fim da resposta.

> *"Runs immediately after a tool completes successfully."*
>
> Documentação oficial do Claude Code · Hooks reference · PostToolUse

O `quality-gate.sh` é o arcabouço de verificação: antes de acusar, separa teste que falhou de container parado ou dependência faltando, para o modelo não entrar em loop consertando o que não é código. No fim, roda o Pint e o `sail artisan test`.

Com o ambiente de pé (`sail up -d`) e os testes verdes, o teste: pedir uma edição que faça o Pint falhar. O `PostToolUse` dispara, o erro volta para o modelo, ele restaura o arquivo, e a nova edição passa.



Pedir, não digitar

O Claude escreve o `settings.json` e o script. O seu trabalho é dizer o que travar e revisar o que voltou.

Evento, matcher, comando

Tudo começa no `settings.json`: qual evento escutar, em quais ferramentas e qual programa rodar.

Ambiente não é código

Container parado não é teste vermelho. Hook que confunde os dois prende o modelo tentando consertar o que não quebrou.

Código sobre código

Estilo pego na edição, teste no fim da resposta, sem depender de lost in the middle nem de context rot. É o que deixa a sessão trabalhar sozinha.

2 O que a aula *não teve tempo de dizer*

ALÉM DA AULA · O "BLOQUEOU" DA TELA

No `PostToolUse` a edição já aconteceu. O `exit 2` e o "blocking error" da tela não desfazem nada: o stderr vai para o modelo, que decide o que fazer (na gravação, restaurou o arquivo). Até lá, o arquivo fica quebrado. Para impedir antes, o evento é o `PreToolUse`, da aula 2.3.

Fonte: Hooks reference, tabela "Exit code 2 behavior per event": PostToolUse, "Shows stderr to Claude; the tool already ran".

ALÉM DA AULA · TRÊS JEITOS DE SAIR, E QUEM ENXERGA CADA UM

SAÍDA DO HOOK	O MODELO VÊ?	QUANDO USAR
`exit 2` + **stderr**	Sim, como erro da ferramenta	Código errado: ele corrige
`exit 0` + `additionalContext`	Sim, como contexto	Ambiente fora: ele sabe, não conserta
`exit 1`	Não. Só você vê um aviso	Nunca num gate

O `exit 1`, convenção do Unix para falha, é o que sai de um script escrito sem pensar no Claude Code. A documentação chama de "non-blocking error": "The action goes ahead", e você vê só o aviso "Failed with non-blocking status code". Aviso no stderr com `exit 0` também não chega: "Claude never sees stderr from a hook that exits 0." Por isso o script usa o `additionalContext`.

Fonte: Hooks reference, "Exit code output", "Exit code 0" e "Other exit codes".

ALÉM DA AULA · COMO O SCRIPT SEPARA AMBIENTE DE CÓDIGO

Pelo código de saída de cada ferramenta e por textos conhecidos. ESLint e Ruff saem com 1 quando acham erro e com 2 quando a configuração quebrou (teste feito: ESLint 10 sem `eslint.config.*` sai com 2). O pytest sai com 3 ou 4 em erro interno ou de uso, e com 5 sem teste coletado (o script trata como ok). 124 é tempo estourado; 126, comando sem permissão de execução; 127, comando não encontrado. `Sail is not running` e banco recusando conexão viram aviso. Hook que trata todo código diferente de zero como erro manda o modelo "corrigir" arquivo certo.

Fontes: ESLint CLI, "Exit Codes"; Ruff, "Exit codes"; pytest, "Exit codes".

ALÉM DA AULA · LINT A CADA EDIÇÃO, TESTE NO FIM DA RESPOSTA

Foi a escolha do Claude na gravação, explicada na tela: teste a cada edição deixaria tudo lento e falharia no meio de refatorações. O `exit 2` no `Stop` faz o modelo continuar. Na volta seguinte chega `stop_hook_active: true`; se ainda falhar, o script libera e avisa você. O Claude Code também tem um freio: 8 bloqueios seguidos sem ferramenta no meio encerram a resposta.

Prefere o erro do teste já na edição? `QG_TESTE_NA_EDICAO=1` (`modelos/settings.teste-a-cada-edicao.json`). O custo: com várias edições de uma vez, o `PostToolUse` dispara em paralelo, um por arquivo.

Fontes: Hooks reference, "Stop input"; Automate actions with hooks, "Stop hook hits the block cap"; Hooks reference, "PostToolBatch".

ALÉM DA AULA · O QUE O MATCHER EDIT|WRITE NÃO VÊ

Arquivo mudado por comando no terminal passa batido: `sed -i`, `php artisan make:*`, script que gera código. A documentação: "Claude Code doesn't run a PostToolUse hook matching Edit|Write when a Bash command or a process outside Claude Code rewrites the same file." O `MultiEdit` da gravação não está na lista de ferramentas do Claude Code 2.1.316: `Edit|Write` basta. E o `timeout`: hook que chega nele é cancelado e a saída é descartada, sem aviso. Por isso o script tem um prazo para o hook inteiro, abaixo do `timeout` (100 s contra 120 na edição, 540 s contra 600 no `Stop`): cada comando ganha só o que sobrou, e o que passa do prazo é interrompido e vira aviso. Interromper exige `timeout` (Linux) ou `gtimeout` (macOS, passo 1).

Fontes: Hooks reference, "PostToolUse" e "Timeouts"; Tools reference.

ALÉM DA AULA · POR QUE O ERRO DO PINT VEIO EM JSON

Na tela, o erro chegou como `{"tool":"pint","result":"fail",...}` porque o script da gravação roda `pint --test --format agent`, o formato curto para agente. O script deste repositório não passa a flag e recebe o mesmo JSON: desde a 1.27, o Pint troca de formato sozinho quando acha `CLAUDECODE` no ambiente (hoje, pelo `laravel/agent-detector`). Teste feito para este material: os hooks recebem essa variável, e o Sail (desde a 1.57) repassa ao container. No Pest e no PHPUnit, isso só acontece com o `laravel/pao`: vem no esqueleto atual do Laravel 13, mas falta em projeto criado antes. Sem ele, o script corta a saída do teste nas últimas 30 linhas. Para instalar:

●●● Na raiz do projeto · Laravel — laravel

```
composer require laravel/pao --dev
# com Sail: ./vendor/bin/sail composer require laravel/pao --dev
```

Fontes: código do Pint 1.27 a 1.32 (ElaborateSummary.php, runningInAgent), do laravel/agent-detector e do bin/sail do Sail 1.57; laravel/pao no Packagist; composer.json do laravel/laravel 13.x.

3 Instale *no seu projeto*

Vai para o seu repositório: `.claude/hooks/quality-gate.sh` e os dois hooks no `.claude/settings.json`. Se você já tem um, o instalador soma os hooks aos seus e guarda uma cópia do original.

1 Confira o jq.

O hook lê o JSON do Claude Code com ele: `jq --version` tem que responder com a versão. Não tem? Instale. No macOS, junto vai o `coreutils`, que traz o `gtimeout` para o script interromper comando travado (no Linux e no WSL, o `timeout` já vem).

Conferir — shell

```
jq --version
```

Ubuntu e WSL — shell

```
sudo apt install -y jq
```

macOS — shell

```
brew install jq coreutils
```

2 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou numa aula anterior? Só atualize:

Terminal · qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

3 Comece do verde, como na aula.

O passo 2 deixou o terminal na pasta pessoal: volte para a raiz do seu projeto, e daqui em diante tudo roda de lá. Como na gravação, suba o ambiente e confira que os testes passam: teste já vermelho faz o `Stop` bloquear por algo que não é da sua mudança.

Na raiz do projeto · Laravel com Sail — laravel

```
./vendor/bin/sail up -d
./vendor/bin/sail test
# sem Sail: php artisan test
```

Na raiz do projeto · Node — node

```
npx vitest run
# com Jest: npx jest
```

Na raiz do projeto · Python — python

```
pytest -q
```

4 Instale os arquivos da aula.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 2.2
```

A saída mostra `criado` ou `somado` para cada arquivo. Um `conflito` quer dizer que você já tinha um `quality-gate.sh`: o da aula fica ao lado, como `.harness-lab-novo`.

5 Confira o que entrou.

Terminal · na raiz do seu projeto — shell

```
ls -l .claude/hooks/quality-gate.sh
jq '.hooks | keys' .claude/settings.json
```

O script precisa ter o `x` de executável, e a lista de eventos precisa ter `PostToolUse` e `Stop`. O `settings.json` ficou assim:

.claude/settings.json — instalado pelo instalar.sh · o mesmo JSON, em menos linhas

```
{ "hooks": {
    "PostToolUse": [{ "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "timeout": 120,
        "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/quality-gate.sh post-edit",
        "statusMessage": "quality gate: conferindo o arquivo editado" }] }],
    "Stop": [{
      "hooks": [{ "type": "command", "timeout": 600,
        "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/quality-gate.sh stop",
        "statusMessage": "quality gate: testes dos arquivos editados" }] }]
} }
```

O `$CLAUDE_PROJECT_DIR` entre aspas acerta o caminho mesmo depois de um `cd` do modelo e com espaço no nome da pasta. O `Stop` não tem matcher: dispara em toda resposta, e o script sai na hora se nada foi anotado.

6 Teste o hook fora do Claude Code.

O hook é só um programa: entra JSON no stdin, sai código. Mande o JSON que o Claude Code mandaria para um arquivo quebrado. Em PHP ou Laravel:

Terminal · na raiz do projeto PHP ou Laravel — shell

```
printf '<?php\necho "oi"\n' > quebrado.php
json=$(jq -nc --arg f "$PWD/quebrado.php" '{tool_input:{file_path:$f}}')
echo "$json" | .claude/hooks/quality-gate.sh post-edit || echo "exit $?"
rm quebrado.php
```

o que aparece — stderr do hook e o exit, com PHP no host

```
quality-gate (PostToolUse): achei problema no código. Corrija antes de seguir:

## php -l quebrado.php

Parse error: syntax error, unexpected end of file, expecting "," or ";" in quebrado.php on line 3
Errors parsing quebrado.php

exit 2
```

Com Sail, o rótulo ganha `(no Sail)` e o erro pode vir como `PHP Parse error:`. Arquivo certo: nada aparece (`exit 0`). Veio um JSON com "parte da checagem não rodou"? O PHP não rodou (Sail

parado, ou nenhum PHP no host): é o aviso de ambiente, com `exit 0`.

Em Node, acusa o ESLint: sem ele, nada aparece; sem `eslint.config.*`, vem o aviso de ambiente. Em Python, o ruff ou, sem ele, a checagem de sintaxe do Python. Nos dois, o esperado é o erro e `exit 2`:

Terminal · na raiz do projeto Node — shell

```
printf 'const x = ;\n' > quebrado.js
json=$(jq -nc --arg f "$PWD/quebrado.js" '{tool_input:{file_path:$f}}')
echo "$json" | .claude/hooks/quality-gate.sh post-edit || echo "exit $?"
rm quebrado.js
```

Terminal · na raiz do projeto Python — shell

```
printf 'def f(:\n    pass\n' > quebrado.py
json=$(jq -nc --arg f "$PWD/quebrado.py" '{tool_input:{file_path:$f}}')
echo "$json" | .claude/hooks/quality-gate.sh post-edit || echo "exit $?"
rm quebrado.py
```

7 **Abra o Claude Code e confira os hooks.**

Aceite a confiança na pasta: antes disso, nenhum hook de settings roda. O `/hooks` deve listar `PostToolUse` e `Stop`, vindos do settings do projeto.

Dentro do Claude Code — sessão

```
/hooks
```

8 **Faça o teste da aula.**

O primeiro pedido é o da gravação; nas outras stacks, use o segundo. Não diga "não corrija": o modelo obedece você, não o hook.

Dentro do Claude Code — sessão

```
> Edite um arquivo PHP de um jeito que faça o Pint falhar
> Crie um arquivo com erro de sintaxe, para a gente testar o hook
```

O esperado: `PostToolUse:Edit hook returned blocking error` (ou `PostToolUse:Write`, se ele criou o arquivo) e o modelo corrigindo o arquivo sozinho.

9 **Teste o Stop.**

O `Stop` só roda o teste ligado ao arquivo: o `<Classe>Test.php` de mesmo nome, ou um teste com `use` da classe. Escolha uma classe que tenha esse teste (num Laravel novo, crie os dois antes) e troque o `Carrinho` pelos seus nomes:

Dentro do Claude Code — sessão

```
> Mude só um valor em app/Services/Carrinho.php que o CarrinhoTest confere
```

O "só" importa: se o modelo atualizar o teste junto, o `Stop` passa sem nada para ver. Senão, o `Stop` roda o `CarrinhoTest` no fim da resposta, e o modelo recebe a falha e continua.

O `quality-gate.sh` é comentado, para ler e adaptar. O que ele roda (ferramenta ausente é pulada):

STACK	A CADA EDIÇÃO	NO FIM DA RESPOSTA
PHP / Laravel	`php -l` e `pint --test`, no Sail se o projeto usa	Pest ou PHPUnit: o `<Classe>Test.php` e os testes com `use` da classe
JS / TS	ESLint	`vitest related --run` ou `jest --findRelatedTests`
Python	`ruff check`, ou checagem de sintaxe sem ruff	pytest: `test_<nome>.py` e `<nome>_test.py`

Em monorepo, ele sobe da pasta do arquivo até o `composer.json`, o `package.json` ou o `pyproject.toml` e roda de lá. Quem decide a resposta é a função `responde()`, no fim do script: erro de código vai no stderr, com `exit 2`; aviso de ambiente vai num JSON com `systemMessage` (e `additionalContext` no `PostToolUse`), com `exit 0`. No `Stop`, o aviso é só seu: `additionalContext` ali reabriria a conversa.

Ajustes sem mexer na lógica

No topo do script, ou como variável na frente do `command` (como no `modelos/settings.teste-a-cada-edicao.json`): `QG_ESTILO` (`acusa`, o padrão, só confere; `corrige` formata sozinho e poupa uma volta), `QG_TESTE_NA_EDICAO=1`, `QG_SAIL` (`auto`, `sim`, `nao`) e os prazos `QG_LIMITE_EDICAO` e `QG_LIMITE_STOP`, que valem para o hook inteiro e ficam abaixo do `timeout`: subiu um, suba o outro. Tabela completa no `README.md` da aula.

4 Arquivos que *reforçam a aula*

`.claude/hooks/quality-gate.sh`

O arcabouço que a aula elogiou, testado: lint na edição, testes no fim da resposta, ambiente separado de código. Laravel (com ou sem Sail), Node, Python e o bash 3.2 do macOS.

No repositório: modulo-2-trava/aula-2.2-o-hook-que-roda-lint-e-teste/projeto/.claude/hooks/

`.claude/settings.json`

"Tudo se inicia a partir do settings.json": evento, matcher e comando. Vai para o git, e o time inteiro recebe a mesma trava.

No repositório: modulo-2-trava/aula-2.2-o-hook-que-roda-lint-e-teste/projeto/.claude/

`modelos/pedir-o-hook.md`

O "saber pedir" por escrito: o pedido da gravação, a versão com critérios e a lista para revisar o hook que o Claude escrever.

No repositório: modulo-2-trava/aula-2.2-o-hook-que-roda-lint-e-teste/modelos/

`modelos/settings.teste-a-cada-edicao.json`

A outra escolha de desenho: sem `Stop`, com o teste rodando a cada edição. O erro chega na hora; cada edição espera o teste.

No repositório: modulo-2-trava/aula-2.2-o-hook-que-roda-lint-e-teste/modelos/

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
Nada acontece depois da edição	Confiança na pasta não aceita, `settings.json` inválido ou script sem `chmod +x`	`/hooks` lista o hook? Se não, o passo 5 aponta a linha do JSON inválido. Se lista, confira o `x` no `ls -l`
Aviso `instale o jq`	jq ausente: o hook sai com 1, você vê o aviso, o modelo segue	Instale (passo 1) e abra outra sessão
`Sail is not running` a cada edição	Containers parados: o hook avisa e não bloqueia	Suba os containers (passo 3), ou `QG_SAIL=nao` se o PHP roda no host
Falha de estilo pede permissão para o Pint	No modo `acusa`, o modelo corrige rodando o Pint no terminal	"Yes, and don't ask again" na primeira vez, ou `QG_ESTILO=corrige`
Pedi para manter o erro e o modelo não corrigiu	O modelo segue o seu pedido. O `Stop` bloqueia uma vez, libera com "ainda falha" e volta a bloquear uma vez em cada resposta seguinte, até o teste passar	Normal. Para ver a correção, peça sem "não corrija", ou depois: "corrija o que o hook apontou"
`Nenhum teste ligado a ...`	O `Stop` acha o teste pelo nome (`<Classe>Test.php`, `test_<nome>.py`) ou, no PHP, pelo `use` da classe. Teste de Feature por rota não conta	Crie o teste com esse nome, ou ponha o `use` da classe nele. Em JS, o Vitest e o Jest seguem os imports
"Não rodou" com ESLint	ESLint 9 ou mais novo sem `eslint.config.*` sai com 2: configuração, não código	Crie o `eslint.config.js`, ou remova o ESLint
Desligar o gate agora	Você quer uma sessão sem hooks, sem apagar nada	O comando logo abaixo da tabela

●●● Terminal · sessão sem nenhum hook — claude

```
claude --settings '{"disableAllHooks": true}'
```

Para ver o que cada hook fez e quanto demorou: `/debug` na sessão liga o log e mostra onde ele fica. As linhas com `Hooks:` trazem evento, comando, código de saída e tempo.

6 Você sai *com*

- [ ] `.claude/hooks/quality-gate.sh` executável e `.claude/settings.json` com `PostToolUse` (`Edit|Write`) e `Stop`.
- [ ] O `/hooks` mostrando os dois, vindos do settings do projeto.
- [ ] O JSON falso devolvendo `exit 2` para um arquivo quebrado e nada para um certo.

- [ ] Numa sessão, o erro de lint voltou sozinho ao modelo, e um teste vermelho no fim da resposta fez ele continuar.
- [ ] A diferença entre `exit 2` (código), `exit 0` com JSON (ambiente) e `exit 1` (o modelo não vê).

PROMETIDO NA AULA

- ✓ "Vamos construir um hook ou dois" e o `quality-gate.sh` na pasta `hooks`: os dois hooks em `projeto/.claude/settings.json` e a versão testada do script em `projeto/.claude/hooks/`, com `instalar.sh 2.2` (seção 3, passo 4).
- ✓ "Não é que o teste falhou, é que o container não está de pé": aviso de ambiente com `exit 0` e `additionalContext`, provado no `teste/verificar.sh` (seção 2).
- ✓ "Ele vai rodar o sail artisan test e o sail pint": o Sail é usado sozinho quando o projeto tem; Node e Python também (seção 3).
- ✓ "O mais importante hoje em dia é saber pedir": o pedido da aula e a versão com critérios em `modelos/pedir-o-hook.md`.
- ✓ O teste "edite um arquivo PHP que faça falhar o Pint": seção 3, passo 8.

7 Glossário

Quality gate	A checagem automática que toda mudança precisa passar para seguir.
PostToolUse	Evento disparado logo depois que uma ferramenta deu certo. Não desfaz a ação.
Stop	Evento do fim da resposta. Com `exit 2`, o modelo continua trabalhando.
Matcher	Filtro do evento. `Edit|Write` é uma lista exata de nomes de ferramenta.
additionalContext	Texto que o hook põe no contexto do modelo, sem ser erro.
systemMessage	Aviso que o hook mostra para você na tela.
stop_hook_active	Chega `true` quando o modelo já continua por causa de um Stop.
CLAUDE_PROJECT_DIR	Raiz onde a sessão começou. Não muda quando o modelo dá `cd`.

8 Para *ir além*

- Hooks reference: "Exit code output", "PostToolUse" e "Stop", o que cada saída faz.
- Automate actions with hooks: exemplos e a seção "Hook not firing".
- Laravel Pint (`--test`) e Laravel Sail (`sail up -d`, `sail test`, `sail php`).
- ESLint, Ruff e pytest: os códigos de saída que separam código errado de ferramenta quebrada.

Próxima: *Aula 2.3 · O hook que bloqueia comando perigoso*

O quality gate pega o erro depois que a edição aconteceu. Agora, um `PreToolUse` que barra antes de rodar: `migrate:fresh`, `push --force`, produção.